IST 387: Introduction to Applied Data Science
Lab 4: Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- Introduction to Sampling
- Sampling in R
- Repeating Our Sampling
- Law of Large Numbers and the Central Limit Theorem
- Comparing Two Samples
- CASE STUDY: Analyzing the Impact of a New Treatment
- Important R Functions

Preeti Jagadev

Introduction

❖ Sampling

In many situations we cannot get access to all the data we would like to have. If we are examining the quality of products coming off a production line or the success of a marketing campaign, in most cases we consider ourselves lucky to get even a small percentage of all the relevant data that might exist. Fortunately, statisticians have worked for a long time on mathematical tools that allow us to make good decisions based on a limited amount of data. The technical term for this is ***sampling.***

We can use data from a sample (a limited subset of all the data) to reason about a population. We can make the concept of sampling clearer with a physical **example:**

Imagine a jar full of gumballs of two different colors, red and blue. The jar was filled from a source that provided 100 red gumballs and 100 blue gumballs, but when they were all poured into the jar, they got mixed. If you drew eight gumballs from the jar at random, what colors would you get?

If things worked out perfectly, which they rarely do, you would get four red and four blue. This is half and half, the same ratio of red and blue that is in the jar. Of course, it rarely works out this way. Instead of getting four red and four blue, you might get three red and five blue, or any other mix you can think of. In fact, it would be possible, though perhaps not likely, to get eight red gumballs. The basic situation, though, is that we really don't know what mix of red and blue we will get with one draw of eight gumballs. That's uncertainty for you, the forces of randomness affecting our sample of eight gumballs in unpredictable ways.

Preeti Jagadev

Here's an interesting idea, that is great at showing what will occur in the long run. Pull eight gumballs from the jar, count the number of red ones, and then throw them back. We do not have to count the number of blue because we can subtract the number of red gum- balls from the total (eight gumballs) to know the number of blue gumballs. Mix up the jar again and then draw eight more gumballs and count the number of red. Repeat this process many times. Here's an example of what you might get:

Draw	#Red
1	5
2	3
3	6
4	2

Notice that the left-hand column is just counting the number of times we drew a sample. The right-hand column is the count of the number of red gumballs in each sample draw. In this example, results are all over the place. In sample draw 4, we only have two red gumballs, but in sample draw 3 we have six red gumballs. But the most interesting part of this example is that if you average the number of red gumballs over all the draws, the average comes out to four red gumballs per draw, which is exactly what we would expect in a jar that is half red and half blue.

Of course, this is a contrived example, and we won't always get such a perfect result so quickly, but if you did 4,000 draws instead of four, you would get close to four red gumballs per draw.

This process of repeatedly drawing a subset from a population is called **sampling**. The population in this example is the whole jar of gumballs. If we do a lot of sampling and record the results each time, the result is called a **sampling distribution**.

Sampling in R

R has a function called **sample( ),** that draws a random sample from a dataset with just a single call. Consider the **dataset from Lab 2 notes:**

```
install.packages("readr")
library(readr)
testFrame <- read_csv("example2.csv")
testFrame$Region <- gsub("\\.", "", as.character(testFrame$Region))
testFrame$change <- testFrame$April.2011 - testFrame$April.2010
testFrame
```

The output is as follows:

```
> testFrame
# A tibble: 14 × 6
   Region           Census Estimate April.2010 April.2011 change
   <chr>             <dbl>    <dbl>      <dbl>      <dbl>  <dbl>
 1 Alabama         4779736  4779735    4785401    4802740  17339
 2 Alaska           710231   710231     714146     722718   8572
 3 Arizona         6392017  6392013    6413158    6482505  69347
 4 Arkansas        2915918  2915921    2921588    2937979  16391
 5 California     37253956 37253956   37338198   37691912 353714
 6 Colorado        5029196  5029196    5047692    5116796  69104
 7 Florida        18801310 18801311   18838613   19057542 218929
 8 Georgia         9687653  9687660    9712157    9815210 103053
 9 New York       19378102 19378104   19395206   19465197  69991
10 North Carolina  9535483  9535475    9560234    9656401  96167
11 Tennessee       6346105  6346110    6357436    6403353  45917
12 Texas          25145561 25145561   25253466   25674781 421315
13 Virginia        8001024  8001030    8023953    8096604  72651
14 Washington      6724540  6724540    6742950    6830038  87088
```

Preeti Jagadev

Further, we can proceed with the following code:

```
set.seed(1)
sample(testFrame$Census, size=5, replace=TRUE)
```

The output is as follows:

```
> set.seed(1)
> sample(testFrame$Census, size=5, replace=TRUE)
[1] 19378102  2915918 18801310  4779736   710231
```

- **set.seed**: This command controls the random number generation process in R so that you can obtain the same results as I did.

- **sample( ):** Code containing functions like **sample( )** that use random numbers can be hard to debug without a way to make the results more predictable. As a matter of good practice, note thar we called the sample( ) function with three arguments:

1. The first argument was the **data source.**
2. For the second and third arguments, we have used named arguments to make sure that R does what we want it to.

   The **size=5** controls the number of observations we want R to sample. The **replace=TRUE** argument specifies a style of sampling called s**ampling with replacement** that statisticians use often to simplify their proofs.

   For us, sampling with or without replacement does not usually have any practical effects, so we will just go with what the statisticians typically do.

We can also ask R to calculate the **mean( )** of a sample, as follows:

```
mean(sample(testFrame$Census, size=5, replace=TRUE))
```

The output is as follows:

```
> mean(sample(testFrame$Census, size=5, replace=TRUE))
[1] 8116642
```

The output no longer shows the five values that R sampled from the complete list. Instead, it used those values to calculate the mean and display that for us. However, when we draw a sample, we will rarely hit the true population mean right on the head, unless we have a very large sample.

Preeti Jagadev

Repeating Our Sampling

When we are trying to understand what happens in the long run, we're not really interested in any one sample or sample mean, but rather in what happens when we replicate the process of sampling many times. So now we've got to get R to repeat this process for us, not once, not four times, but 400 or 4,000 times.

Like most programming languages, R has a variety of ways of repeating an activity. One of the easiest ones to use is the **replicate( )** function. To start, let's just try four replications:

```
replicate(4, mean(sample(testFrame$Census, size=5, replace=TRUE)))
```

The output is as follows:

```
> replicate(4, mean(sample(testFrame$Census, size=5, replace=TRUE)))
[1] 18405154  9925202 26528444 18295633
```

We took the exact same command as before, which was a nested function to calculate the **mean( )** of a random sample of states. This time, we put that command inside the **replicate( )** function so we could run it over and over again.

Now we will encapsulate the whole replicate function inside another call to **mean( ),** so that we can get the average of 400 of the sample means. Here we go:

```
mean(replicate(400, mean(sample(testFrame$Census, size=5, replace=TRUE))))
```

The output is as follows:

```
> mean(replicate(400, mean(sample(testFrame$Census, size=5, replace=TRUE))))
[1] 11765045
```

This deeply nested command accomplishes the following:

- Draws 400 samples of size n=5 from our full data set.
- Calculates the mean from each sample and keeps it in a list.
- When finished with the list of 400 of these means, calculates the mean of that list of means.
- The resulting value is still not the exact value of the mean of the whole data set, but it is getting much closer.

Let's push it further and see if we can get closer to the true mean for all our data:

```
mean(replicate(4000, mean(sample(testFrame$Census, size=5, replace=TRUE))))
```

The output is as follows:

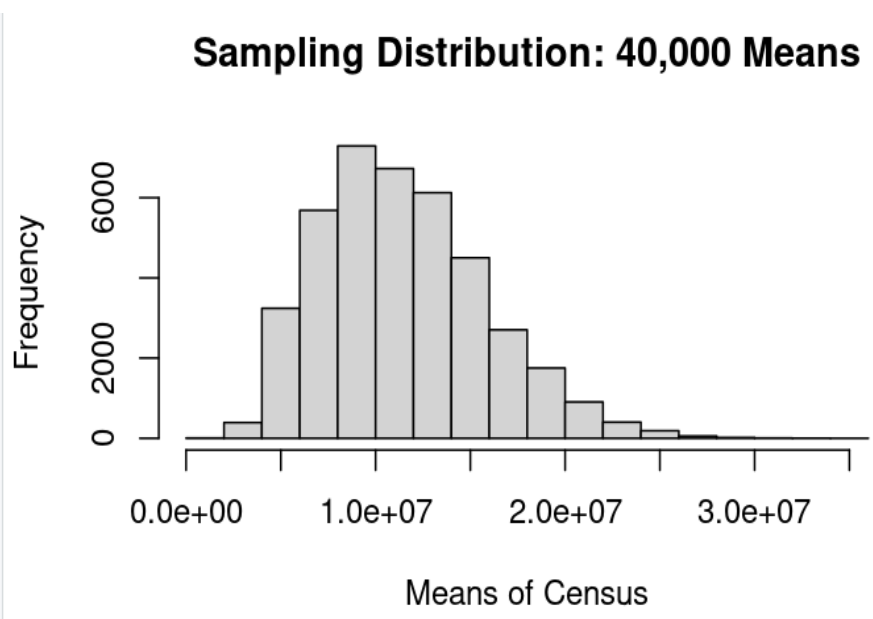
```
> mean(replicate(4000, mean(sample(testFrame$Census, size=5, replace=TRUE))))
[1] 11549731
```

Instead of summarizing our whole sampling distribution into a single average, let's look at a distribution of means using a histogram:

Preeti Jagadev

```
hist(replicate(40000, mean(sample(testFrame$Census, size=5, replace=TRUE))),
     main="Sampling Distribution: 40,000 Means",
     xlab="Means of Census")
```

The output is as follows:



The histogram displays a list of 40,000 means as frequencies. The histogram shows a very typical configuration that is almost bell-shaped, although it has a bit of skewness off to the right.

Law of Large Numbers and the Central Limit Theorem

1. If you run a statistical process many times, it will tend to converge on a stable result (**the law of large numbers**). In the previous example we wanted to know how many samples, of size n=5 we would have to draw before we could get a good approximation of the true value. We learned that drawing one sample provided a poor result, while drawing 4,000 samples gave us better results.

2. When we are looking at sample means, and we take the law of large numbers into account, we find that the distribution of sampling means starts to create a bell-shaped or normal distribution, and the center of that distribution (the mean of all those sample means) gets really close to the actual population mean. It gets closer faster for larger samples; in contrast, for smaller samples you must draw lots and lots of them to get close.

Let's illustrate this with the following example:

```
mean(replicate(100, mean(sample(testFrame$Census, size=14, replace=TRUE))))
```

The output is as follows:

```
[1] 11228477
```

Preeti Jagadev

Here's a run that repeats only 100 times, but each time draws a sample of n=14 (equal in size to the population). You might be scratching your head now, saying, "Wait a minute. Isn't a sample of 14 the same thing as the whole list of 14 observations?" This does seem confusing, but it goes back to the question of sampling with replacement, and that appears in the command above as replace-TRUE).

Sampling with replacement means that as you draw our one value to include in your random sample, you immediately chuck it back into the list so that potentially, it could get drawn again either immediately or later.

We can go even higher in our sample size with no trouble:

```
mean(replicate(100, mean(sample(testFrame$Census, size=5200, replace=TRUE))))
```

The output is as follows:

```
> mean(replicate(100, mean(sample(testFrame$Census, size=5200, replace=TRUE))))
[1] 11499171
```

The command runs 100 replications using samples of size n=5200. Now the mean of the sampling distribution is close to the population mean! Remember that this result will change a little every time you run the procedure, because different random samples are being drawn for each run. **But the rule of thumb is that the bigger your sample size, what we're calling "n," the closer your estimate will be to the true value. Likewise, the more trials you run, the closer your population estimate will be.**

Now, lets save a distribution of sample means to a new vector called SampleMeans. We should have 10,000 of them:

```
set.seed(1)
SampleMeans <- replicate(10000, mean(sample(testFrame$Census, size=8, replace=TRUE)))
length(SampleMeans)
```

The output is as follows:

```
[1] 10000
```

The mean of all those means should be close to our population mean:

```
mean(SampleMeans)
```

The output is as follows:

```
> mean(SampleMeans)
[1] 11507909
```

Let's look at the summary:

```
summary(SampleMeans)
```

The output is as follows:

```
> summary(SampleMeans)
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
 2750536  8956408 11221372 11507909 13739437 26316353
```

Preeti Jagadev

There is a slightly different way of getting the same information from R that will prove more flexible for us in the long run:

```
quantile(SampleMeans, probs=c(0.25, 0.50, 0.75))
```

The output is as follows:

```
> quantile(SampleMeans, probs=c(0.25, 0.50, 0.75))
     25%      50%      75%
 8956408 11221372 13739437
```

One reason to use **quantile( )** is that it lets us control exactly where we make the cuts. To get quartiles, we cut at 25%, at 50% and at 75%. But what if we wanted instead to cut at the more extreme levels of 2.5% and 97.5%? Easy to do with the **quantile( ).**

```
quantile(SampleMeans, probs=c(0.025, 0.975))
```

The output is as follows:

```
> quantile(SampleMeans, probs=c(0.025, 0.975))
   2.5%    97.5%
5732748 18978417
```

Comparing Two Samples

Consider a sample showing the number of people in eight different areas, where each of these areas is a unit associated with a country (e.g. a province, district, or region):

2238603 2662480 1632934 822441 1955577 1084979 637026 3388768

We can easily get these into R and calculate the sample mean:

```
MysterySample <- c(2238603, 2662480, 1632934, 822441, 1955577, 1084979, 637026, 3388768)
mean(MysterySample)
```

The output is as follows:

```
> mean(MysterySample)
[1] 1802851
```

The mean of our mystery sample is 1,802,851. Here's the question: is this a sample of U.S regions or is it something else? Looking at it all by itself it would be hard to tell. Some of these observations have larger values than some U.S. states and some do not.

We have the sampling distribution of means, and if the new mean is way out in the extreme areas of the sample distribution, then it seems much less likely that our Mystery Sample is a sample of U.S. regions. In this case, we can see quite clearly that 1,802,851 is on the extreme low end of the sampling distribution.

Preeti Jagadev

Recall that when we ran the **quantile( )** command we found that only 2.5% of the sample means in the distribution were smaller than 5732748. In fact, we could even play around with a more stringent criterion:

```
quantile(SampleMeans, probs=c(0.005, 0.995))
```

The output is as follows:

```
> quantile(SampleMeans, probs=c(0.005, 0.995))
    0.5%    99.5%
 4607857 21521297
```

This **quantile( )** command shows that only 0.5% of all the sample means are lower than 4607857. So, our MysterySample mean of 1,802,851 would be a very rare event, if it were truly a sample of U.S. regions. From this we can infer, tentatively but based on good statistical evidence, that our MysterySample is not a sample of U.S. regions.The mean of MysterySample is just too small to be a likely member of our sampling distribution.

This method of reasoning is the basis for one of the most common methods of statistical inference. You construct a comparison distribution based on some statistical values, you mark off a zone of extreme values, and you compare any new sample of data you get to the comparison distribution to see if it falls in the extreme zone. If it does, you tentatively conclude that the new sample was obtained from some source other than what you used to create the original comparison distribution.

In the commands above, we first looked at the mean of the sampling distribution with **mean( )** and we looked at its shape with **hist( ).** Now, lets quantify the spread of the distribution:

```
sd(SampleMeans)
```

The output is as follows:

```
> sd(SampleMeans)
[1] 3454124
```

This shows us the standard deviation of the distribution of sampling means. Statisticians call this the standard error of the mean.

There is a shortcut to finding out the standard error:

```
sd(testFrame$Census)/sqrt(5)
```

The output is as follows:

```
> sd(testFrame$Census)/sqrt(5)
[1] 4535025
```

The formula in this command takes the standard deviation of the original state data and divides it by the square root of the sample size (we have chosen a sample size of n=5). Here **sqrt( )** is the abbreviation for square root. You might notice that the number we got with the shortcut was slightly different than the number that came from the distribution itself, but the difference is not meaningful (and only arises because of randomness in the sampling distribution). Another thing you might have noticed is that the **larger the sample size, the smaller the standard error.**

Preeti Jagadev

One last thing to learn is another shortcut:

We found out the 97.5% cut point by constructing the sampling distribution and then using quantile to tell us the actual cuts. You can also calculate cut points just using the mean and the standard error. Roughly speaking, two standard errors down from the mean is the 2.5% cut point, and two standard errors up from the mean is the 97.5% cut point.

```
stdError <- sd(testFrame$Census)/sqrt(5)
CutPoint975 <- mean(testFrame$Census) + (2*stdError)
CutPoint975
```

The output is as follows:

```
> CutPoint975
[1] 20548681
```

This value is different from what we calculated with the **quantile( )** function. The differences arise because of the randomness in the distribution that we constructed.

CASE STUDY: Analyzing the Impact of a New Treatment

To help improve Southeast's airline's NPS value, at selected airports, prior to boarding the flight, free drinks were provided to passengers. These customers were then surveyed at the end of their

flight. We have access to these surveys and need to figure out if this change improved Southeast's NPS.

```
library(tidyverse)
smallSurvey <- read_csv("smallSurvey.csv")

#Remove the NAs from the LTR column:
smallSurvey <- filter(smallSurvey, !is.na(LTR))

#Read in the new survey results after the treatment
newTreatmentDF <- read_csv("newTreatmentSurvey.csv")

#Check the number of rows:
nrow(newTreatmentDF)
```

The output is as follows:

```
> nrow(newTreatmentDF)
[1] 120
```

Let's look at the NPS values: (Refer Lab 3 Part 1 notes for the CalcNPS(ltr) function. This function is also given at the end of these notes*):

```
CalcNPS(smallSurvey$LTR)
```

Preeti Jagadev

The output is as follows:

```
> CalcNPS(smallSurvey$LTR)
[1] 9.018036
```

And

```
CalcNPS(newTreatmentDF$LTR)
```

The output is as follows:

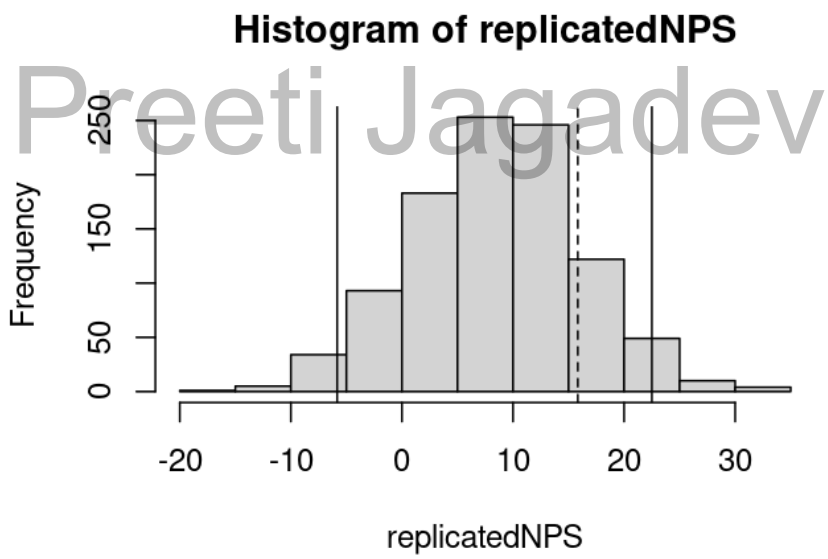
```
> CalcNPS(newTreatmentDF$LTR)
[1] 15.83333
```

So, we can see that the NPS was higher among the participants of the survey after the treatment. Maybe this was just due to the sampling error, or it may have been due to the impact of the treatment on Southeast's customers?

Let's explore:

```
set.seed(100)
#using n=120 to match the number of
#observations in the treatment survey.
replicatedNPS <- replicate(1000,
                 CalcNPS(sample(smallSurvey$LTR, size =120, replace = TRUE)))

#Calculate the 2.5% and 97.5% quantiles
thresholdValues <- quantile(replicatedNPS, probs=c(0.025,0.975))

#Generate a histogram of NPS values:
hist(replicatedNPS)

#Mark the 95% central region:
abline(v=min(thresholdValues))
abline(v=max(thresholdValues))

#Show the new NPS value (linetype is a dashed line)
newTreatmentNPS <- CalcNPS(newTreatmentDF$LTR)
abline(v=newTreatmentNPS, lty="dashed")
```

The output is as follows:



Preeti Jagadev

Here the two solid lines mark the 2.5% and 97.5% quantiles, and the dashed line marks the position of the newTreatmentNPS.

Note where the dashed line falls on the histogram. Even though 15.83 (NPS for New treatment) is larger than 9.01 (the NPS for the original survey), we see that 15.83 falls within the "**central region**" of the histogram. Even though our result of 15.83 looked promising, the randomness that we have to deal with when working with samples shows that this is probably not a credible level of improvement.

*** CalcNPS(ltr) function:**

```
#Calculate NPS
#ltr: A vector of likelihood-to-recommend values
#neutral: The range below which someone is considered
#a detractor and above which someone is considered
#a promoter (default is 7 to 8)

CalcNPS <- function(ltr, neutral=c(7,8)) {

#Create a new vector with just the promoters
#then calculate the length of the promoters vector
promoters <- ltr[ltr > max(neutral)]
numPromoters <- length (promoters)

#Calculate the number of detractors
detractors <- ltr[ltr < min (neutral)]
numDetractors <- length (detractors)

#Calculate NPS
total <- length(ltr)
nps <- (numPromoters/total - numDetractors/total)*100
return(nps)
}
```

Important R Functions

Preeti Jagadev

- **filter( )**: Tidyverse function to subset rows.
- **length( ):** The number of elements in a vector.
- **mean( ):** The arithmetic mean or average of a set of values.
- **nrow( ):** Number of rows in a data structure.
- **quantile( ):** Calculates cut points based on percentages/proportions.
- **replicate( ):** Runs an expression/calculation many times.
- **sample( ):** Chooses elements at random from a vector.
- **sd( ) :** Calculates standard deviation.
- **sqrt( ):** Calculates square root.
- **summary( ):** Summarizes contents of a vector.